

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



MINI PROJECT REPORT
ON
API Based Weather Dashboard
Submitted in the partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(CYBERSECURITY)

Submitted By,
KAVYA P M 01SU23CB013
PRAJNA C ACHAR 01SU23CB021

Under the guidance of
Mr. Yogesh Bandewar

IBM lecturer
Dept. of CyberSecurity

2025-2026

DEPARTMENT OF CYBERSECURITY
SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



Department of Cybersecurity

CERTIFICATE

This is to certify that the Technical Seminar entitled ***“API Based Weather Dashboard”*** is a bonafide work carried out by ***Kavya P M and Prajna C Achar bearing the USN 01SU23CB013 and 01SU23CB021*** in the partial fulfillment for the award of **Bachelor of Technology in Computer Science Engineering (Cyber Security)** of the **Srinivas University- Institute of Engineering & Technology** during the year **2025-26**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The mini project report has been approved as it meets the academic criteria outlined for the said degree.

Name & Signature

Name & Signature

Mr. Yogesh Bandewar
(Project Coordinator)

Prof. Gaurav Prasad
(HOD, Cyber security)

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



Department of Cybersecurity

DECLARATION

We , *Kavya P M and Prajna C Achar*, the students of 6th semester, **Bachelor of Technology** in (Computer Science Engineering)Cyber Security of the **Srinivas University- Institute of Engineering & Technology**, Mukka, hereby declare that the mini project report entitled “*API Based Weather Dashboard*” has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology** in (Computer Science Engineering)Cyber Security of Srinivas University- Institute of Engineering & Technology and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: 23-03-2026

Place: Mukka

ABSTRACT

The **API-Based Weather Dashboard** is a full-stack web application developed using a microservices architecture that emphasizes modularity, scalability, and efficient data handling without relying on a traditional database. The system is composed of multiple independent backend services, including an API Gateway, Weather Service, and Location Service, each responsible for specific functionalities and communicating through HTTP protocols. The API Gateway acts as a central entry point, routing client requests to appropriate services while maintaining separation of concerns.

The Weather Service retrieves real-time weather data such as temperature, humidity, and atmospheric conditions from an external API, while the Location Service converts user-input city names into geographical coordinates required for accurate data fetching. The frontend is designed using HTML, CSS, and JavaScript, providing a responsive and user-friendly interface with features such as search functionality, theme toggling, and local storage-based search history.

By eliminating database dependency and leveraging external APIs along with browser storage, the project simplifies data management while still delivering dynamic and real-time results. This implementation demonstrates key microservices principles such as independent deployment, loose coupling, and service scalability, making it suitable for modern web application development and real-world scenarios.

Keywords: Microservices Architecture, API Gateway, Real-Time Weather Data, OpenWeatherMap API, RESTful Services, Local Storage

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of my mini project.

First and foremost, our sincere thanks go to **Mr. Yogesh Bandewar** , Mini Project Coordinator, for coordinating the project process, offering timely suggestions, and being a constant source of motivation and assistance.

we are immensely grateful to **Prof. Gaurav Prasad**, Head of the Department of Cyber security, for his constant encouragement and valuable support throughout the course of my studies.

We would also like to extend my heartfelt thanks to **Dr. Ramakrishna N Hegde**, Dean, SUIET for his guidance and for facilitating the smooth conduct of the Mini Project Demonstration.

We m deeply thankful to the Management of **Srinivas University Institute of Engineering and Technology** for providing me with the opportunity to undertake this project as part of the academic curriculum. Their vision and commitment to practical learning have made this experience possible.

We would also like to acknowledge the support and encouragement of all the faculty members of the Department of Cyber Security, whose teachings laid the foundation for this practical learning.

Last but not least, we thank my family and friends for their continuous moral support and encouragement during the entire period of the project and report preparation.

This Mini Project has been a truly enriching experience and a vital step in our professional development.

Kavya P M -01SU23CB013
Prajna C Achar-01SU23CB021

TABLE OF CONTENTS

Chapter 1	INTRODUCTION	10
1.1	Introduction to the project	10
Chapter 2	OBJECTIVES	11
2.1	Primary objectives	11
2.2	Secondary objectives	11
Chapter 3	SYSTEM ARCHITECTURE	12
3.1	API Gateway	12
3.2	Weather service	12
3.3	Location service	12
3.4	Front-end	12
3.5	Communication flow	12
Chapter 4	TECHNOLOGY STACK	13
Chapter 5	PROJECT STRUCTURE	14
Chapter 6	FEATURES OF THE SYSTEM	15
CHAPTER 7	ADVANTAGES & LIMITATIONS	16
7.1	Advantages	16
7.2	Limitations	16
CHAPTER 8	FUTURE ENHANCEMENT	17
CHAPTER 9	CONCLUSION	18
CHAPTER 10	OUTPUT	19
CHAPTER 11	REFERENCES	21

LIST OF FIGURES

FIGURE NO	TITLE OF FIGURE	PAGE NO
Fig 1	Weather dashboard	19
Fig 2	Weather service	19
Fig 3	Location service	20
Fig 4	API gateway	20

CHAPTER 1

INTRODUCTION

1.1 Introduction

In modern web development, applications are increasingly designed using scalable and modular architectures. One such approach is the microservices architecture, where applications are divided into smaller, independent services that communicate with each other. This project, titled API-Based Weather Dashboard, demonstrates the practical implementation of a microservices-based system without relying on a traditional database.

The primary goal of this project is to build a responsive and efficient weather dashboard that fetches real-time weather data from external APIs. The system is designed to ensure scalability, maintainability, and clear separation of concerns. Instead of storing data locally in a database, the application dynamically retrieves data from external services and uses browser storage for temporary data.

This project provides hands-on experience in building distributed systems, integrating APIs, and designing clean architectures suitable for real-world applications.

CHAPTER 2

OBJECTIVES

2.1 Primary objectives

- To design and develop a microservices-based weather dashboard.
- To implement independent backend services such as API Gateway, Weather Service, and Location Service.
- To fetch and display real-time weather data using external APIs.
- To ensure proper communication between services using HTTP protocols.
- To build a responsive and user-friendly frontend interface.

2.2 Secondary objectives

- To eliminate the need for a database by using dynamic API data fetching.
- To store temporary user data (search history) using browser localStorage.
- To demonstrate modularity and scalability in application design.
- To implement clean architecture with structured folders (controllers, routes, config).
- To provide additional features like theme switching (dark/light mode).
- To enhance understanding of real-world microservices implementation.

CHAPTER 3

SYSTEM ARCHITECTURE

The system follows a microservices architecture consisting of multiple independent components:

3.1 API Gateway

The API Gateway acts as the central entry point for all client requests. It routes incoming requests to the appropriate backend services. This layer simplifies client interaction and abstracts the complexity of multiple services.

3.2 Weather Service

The Weather Service is responsible for fetching real-time weather data such as temperature, humidity, and weather conditions. It communicates with an external API (OpenWeatherMap) to retrieve accurate and up-to-date information.

3.3 Location Service

The Location Service converts city names into geographical coordinates (latitude and longitude). These coordinates are required by the Weather Service to fetch weather data.

3.4 Frontend

The frontend is built using HTML, CSS, and JavaScript. It provides an interactive user interface where users can search for weather information and view past searches.

3.5 Communication Flow

1. User enters a city name in the frontend.
2. Request is sent to the API Gateway.
3. API Gateway forwards the request to the Location Service.
4. Location Service returns coordinates.
5. API Gateway sends coordinates to the Weather Service.
6. Weather Service fetches weather data from the external API.
7. Response is sent back to the frontend via the API Gateway.

This architecture ensures loose coupling and independent functionality of each service.

CHAPTER 4

TECHNOLOGY STACK

The project uses the following technologies:

4.1 Backend:

Node.js for server-side development

Express.js for building REST APIs

4.2 Frontend:

HTML for structure

CSS for styling

JavaScript for functionality

4.3 External API:

OpenWeatherMap API for real-time weather data

4.4 Storage:

Browser localStorage for storing recent searches

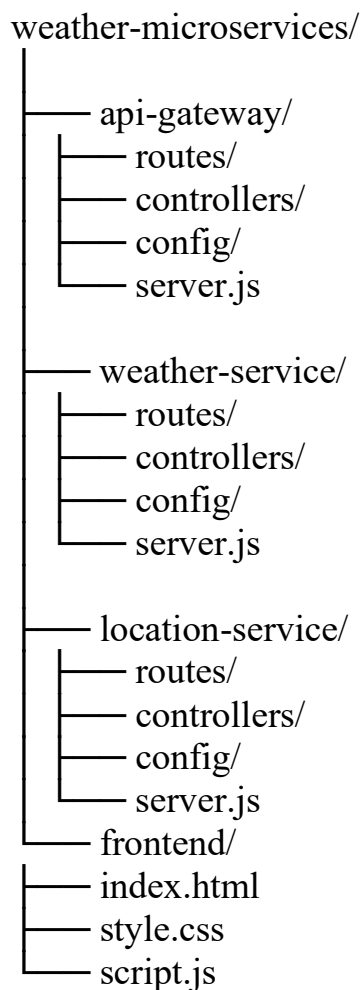
This combination of technologies ensures a lightweight and efficient system.

CHAPTER 5

PROJECT STRUCTURE

5.1 Structure

The project follows a well-organized folder structure to maintain modularity and scalability



Each microservice is self-contained and follows a consistent design pattern, making it easy to maintain and scale.

CHAPTER 6

FEATURES OF THE SYSTEM

The API-Based Weather Dashboard includes the following features:

- Real-time weather data retrieval
- City-based search functionality
- Responsive and modern UI design
- Dark and light theme support
- Search history stored using localStorage
- Modular backend services
- Centralized request handling via API Gateway

These features enhance user experience and demonstrate the effectiveness of microservices architecture.

CHAPTER 7

ADVANTAGES & LIMITATIONS

7.1 Advantages

- Scalability: Each service can be scaled independently.
- Maintainability: Code is organized into separate modules.
- Flexibility: Easy to modify or replace individual services.
- No Database Dependency: Reduces complexity and cost.

7.2 Limitations

- Increased complexity in communication between services
- Dependency on external APIs for data
- No persistent backend storage
- Slight latency due to multiple service calls

Despite these limitations, the system effectively demonstrates modern application architecture.

CHAPTER 8

FUTURE ENHANCEMENT

The API-Based Weather Dashboard successfully demonstrates the implementation of a microservices-based system without a database. By integrating external APIs and using an API Gateway, the system achieves modularity, scalability, and efficient data handling.

Future Enhancements:

- Add user authentication system
- Integrate a database for persistent storage
- Implement caching to improve performance
- Deploy services using Docker and Kubernetes
- Add more weather details such as forecasts and air quality index

This project serves as a strong foundation for understanding microservices and can be extended into a production-level application with additional features and optimizations.

CHAPTER 9

CONCLUSION

The API-Based Weather Dashboard project demonstrates the implementation of a full-stack application using a microservices architecture without a database. The system is divided into independent services such as the API Gateway, Weather Service, and Location Service, ensuring modularity and clear separation of concerns. The API Gateway acts as a central entry point and efficiently manages communication between the frontend and backend services.

The project successfully integrates external APIs to fetch real-time weather data, providing accurate and up-to-date information. The frontend offers a responsive and user-friendly interface with features like search functionality, theme switching, and local storage-based history tracking. By avoiding database usage, the system simplifies data handling while still delivering dynamic results.

This project highlights important concepts such as scalability, flexibility, and independent service deployment. It also provides practical experience in RESTful communication, API integration, and structured project design. Overall, the project serves as a strong foundation for understanding modern web development and microservices architecture.

With further enhancements like database integration, authentication, and deployment tools, this system can be extended into a production-level application.

CHAPTER 10

OUTPUT

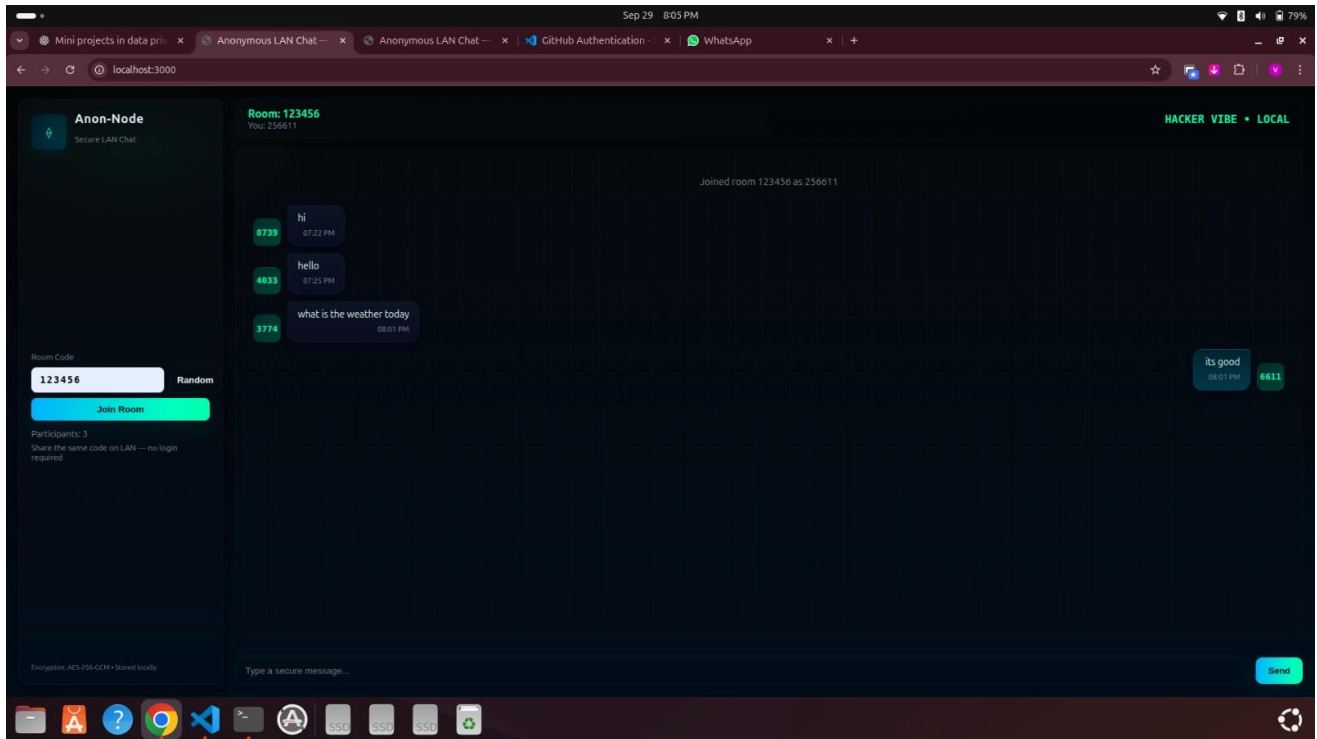


Fig 1 : Weather Dashboard

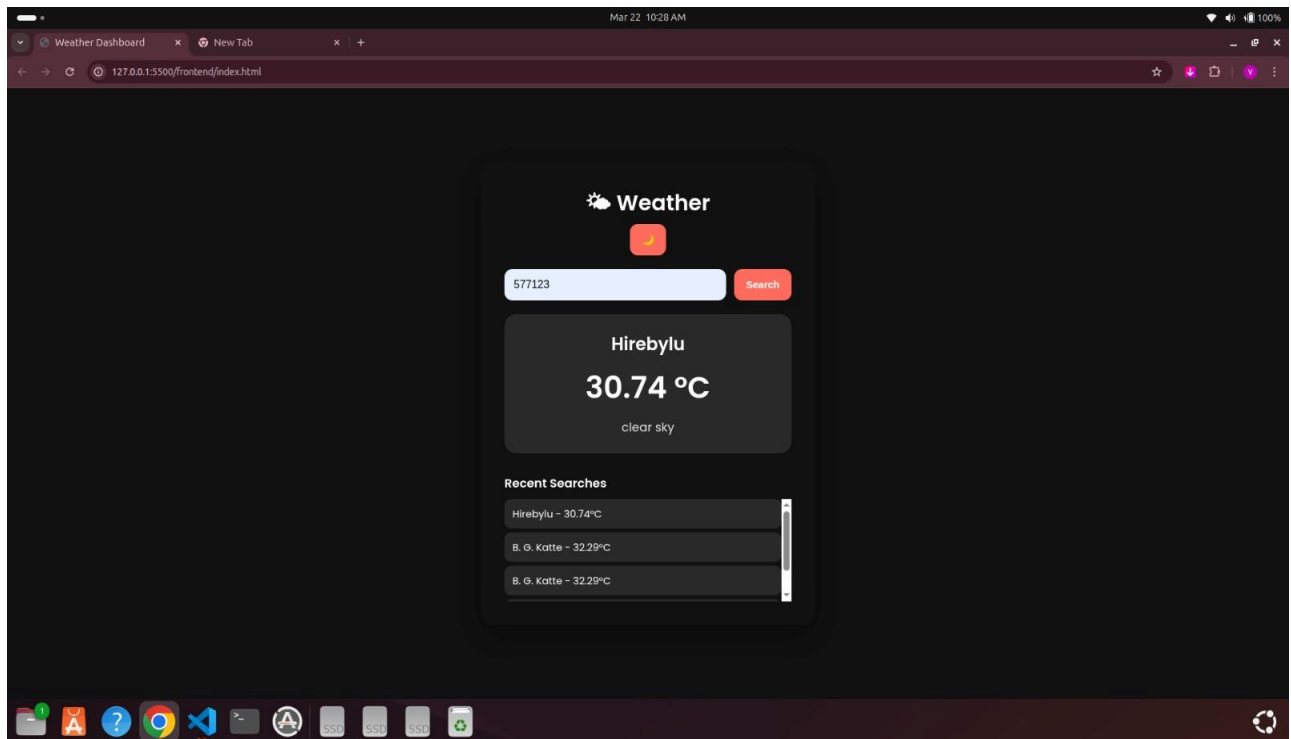


Fig 2 : Weather Service

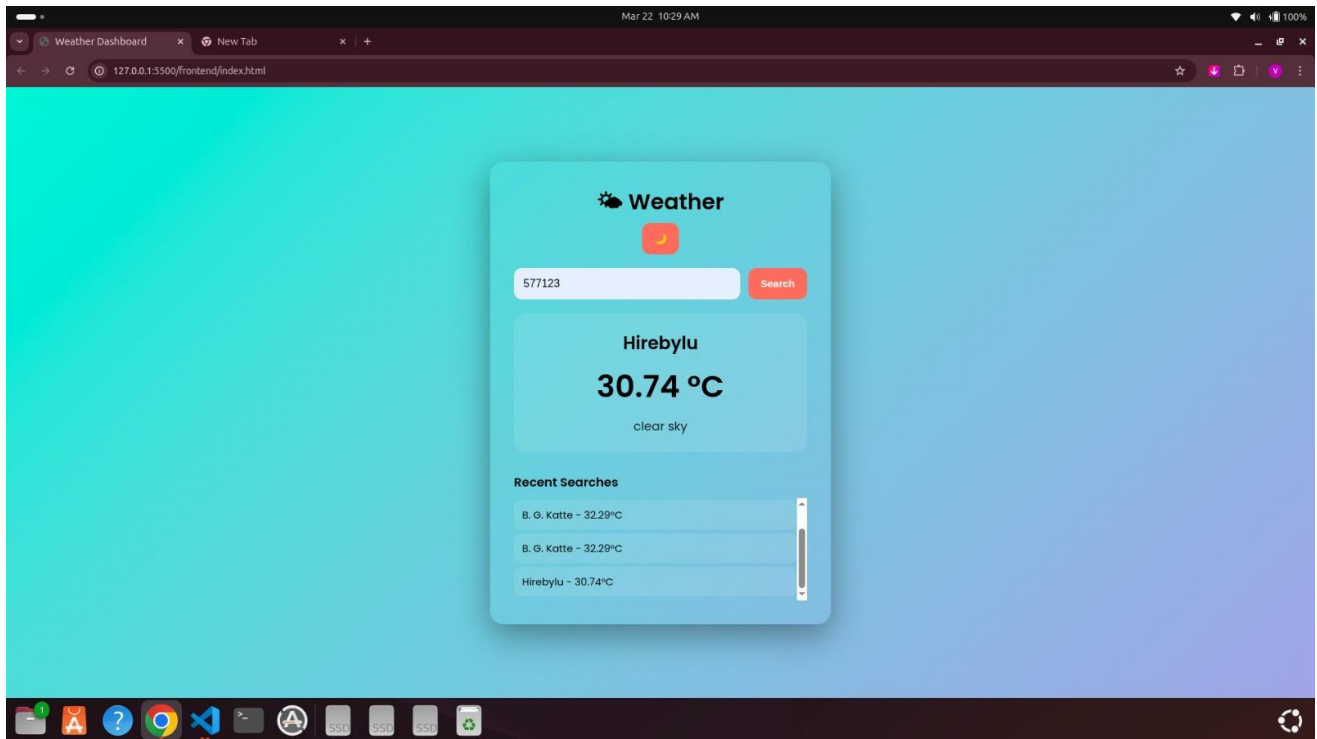


Fig 3 : Location service

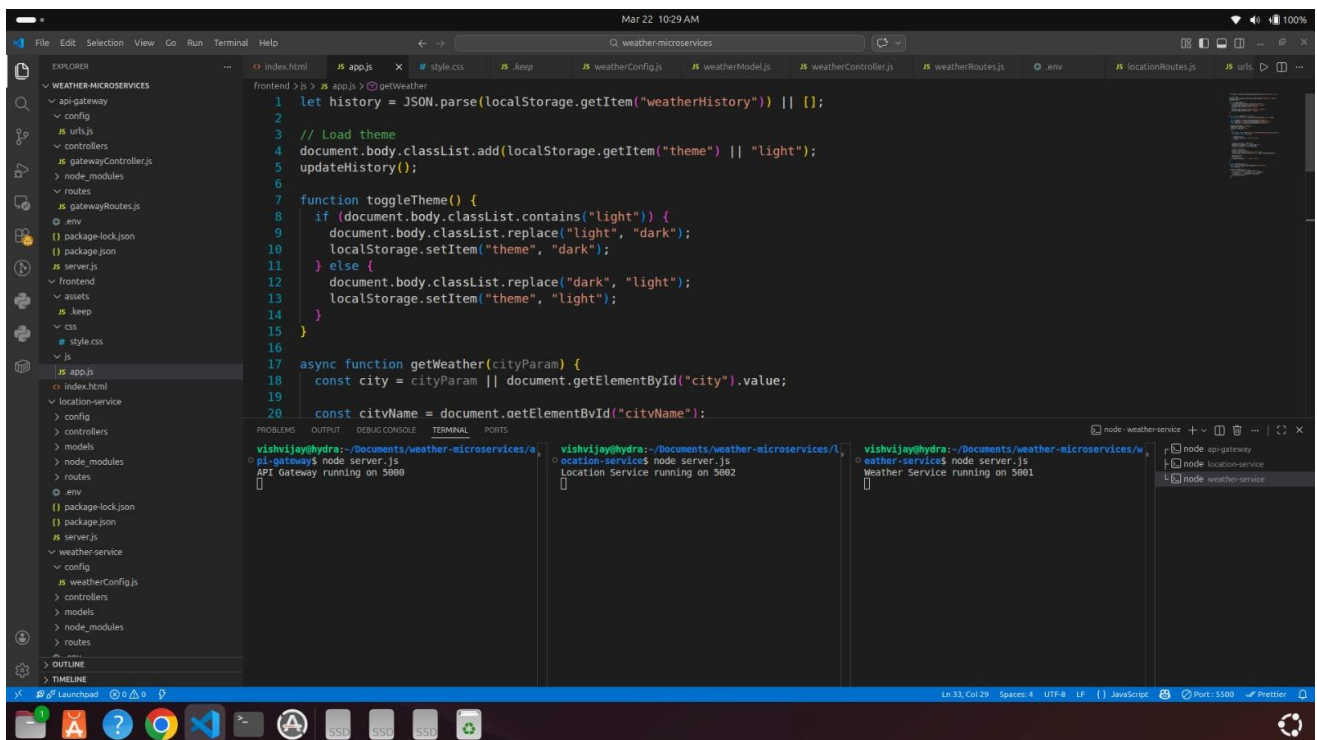


Fig 4 : API gateway

CHAPTER 11

REFERENCES

1. OpenWeatherMap API Documentation

[OpenWeatherMap API Docs](#)

(Provides real-time weather data APIs including current weather, forecast, and geolocation services.)

2. Node.js Official Documentation

[Node.js Documentation](#)

(Covers server-side JavaScript runtime, modules, and API development concepts.)

3. Express.js Official Documentation

[Express.js Docs](#)

(Explains building RESTful APIs, routing, middleware, and backend services.)

4. Guide to Building RESTful APIs with Node.js

[REST API Guide](#)

(Provides understanding of API design, documentation, and implementation practices.)

5. Microservices Architecture Concepts (Node.js Based)

[Microservices with Node.js PDF](#)

(Explains microservices principles such as service decomposition, scalability, and independent deployment.)